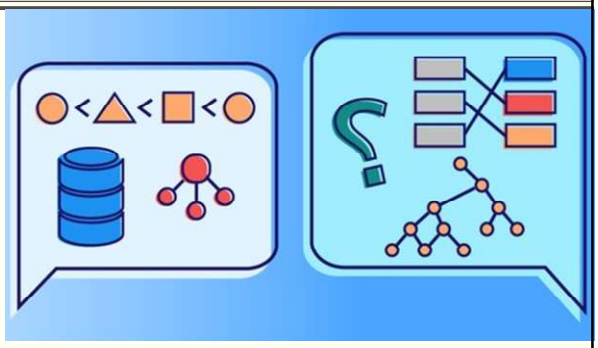
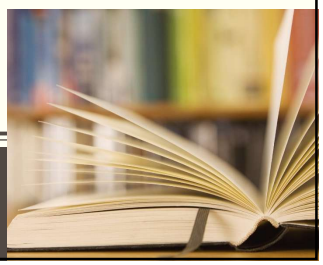
ALGORITMOS Y ESTRUCTURAS DE DATOS



Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 05/10

- Estructuras de datos dinámicas.
- Asignación dinámica de memoria
- Reserva dinámica de memoria.
- Estructuras auto referenciadas.
- Lista enlazada: Agregar un elemento al final, agregar un elemento al principio.
- Lista enlazada: Borrar el primer elemento.
- Lista enlazada: Ejemplo
- Otras listas

2

Algoritmos y Estructuras de Datos – AEDD – Asistencia 05/10

Contraseña del estudiante

h9zsg1

3

Estructuras de datos dinámicas

Estructuras de datos que crecen y se achican durante la ejecución de un programa. Son estructuras de datos dinámicas:

Arrays dinámicos

- Se crean en tiempo de ejecución, a diferencia de los arrays estáticos que se crean en tiempo de compilación.
- Se les asigna memoria recién al ejecutarse su declaración.

Listas enlazadas

- Colección de elementos dispuestos uno a continuación del otro.
- Permiten inserciones y borrados en cualquier parte de la lista.

Pilas

- Permiten inserciones y borrados sólo en el tope (parte superior) de la pila.

Colas

- Permiten inserciones desde un extremo y borrados en el otro.
- Específicamente, las inserciones se hacen en su parte final, y las eliminaciones en su parte inicial..

4

Estructuras de datos dinamicas

- **Según donde se almacenan**
  - **Internas** (en memoria principal)
  - **Externas** (en memoria auxiliar)
- **Según los tipos de datos de sus componentes**
  - **Homogéneas** (todas del mismo tipo)
  - **No homogéneas** (pueden ser de distinto tipo)
- **Según la implementación**
  - **Provistas por los lenguajes** (básicas)
  - **Abstractas** (TDA - Tipo de dato abstracto que puede implementarse de diferentes formas)
- **Según la forma de almacenamiento**
  - **Estáticas** (ocupan posiciones fijas y su tamaño nunca varía durante todo el módulo)
  - **Dinámicas** (su tamaño varía durante el módulo y sus posiciones también)

5

Asignación dinámica de memoria

- Posibilita hacer un uso más eficiente de la memoria disponible, evitando que las variables y estructuras "ocupen espacio" cuando no se usan.
- El espacio de las variables asignadas dinámicamente se crea durante la ejecución del programa, y no como en el caso de las variables estáticas, cuyo espacio se asigna en tiempo de compilación.
- El límite para dicha asignación puede ser tan grande como la cantidad de memoria física disponible en la computadora.
- La asignación se realiza con la instrucción new y el tipo de dato para el que se quiere reservar memoria.

Ejemplo:

```
int *Ptr = new int;
```

Tipo de la variable dinámica a crear

6

Asignación dinámica de memoria

- La memoria dinámica se obtiene y libera durante la ejecución del programa.
- La memoria dinámica la debe gestionar el programador. Cuando necesite crear una variable dinámica deberá solicitar memoria dinámica con el operador:

new :

- Solicita reserva de memoria de un número de bytes.
- Retorna un puntero del tipo de dato para el que se solicita memoria.
- Es un puntero a la dirección del bloque asignado de memoria.
- Si no hay memoria disponible, retorna NULL.
- La asignación dinámica de memoria es muy simple: una única expresión realiza todo el trabajo de cálculo de tamaño y asignación.

7

Asignación dinámica de memoria

- La asignación se realiza con la instrucción new y el tipo de dato para el que se quiere reservar memoria.
- Si la asignación falla, devuelve un NULL.

Una forma de averiguar si falló la asignación de memoria:

```
a = new int;
if (a == NULL){
    cout << "ERROR, no hay memoria suficiente";
}
else{
    ..............
}
```

Otra forma, más elegante:

```
if ((a = new int) == NULL){
    cout << "ERROR, no hay memoria suficiente";
}
else{
    ..............
}
return 0;
```

8

Asignación dinámica de Memoria: Ejemplos

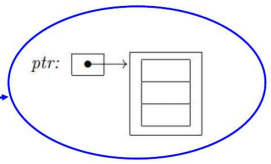
- Ejemplo 1:

```
ptr = new int;
```

- Ejemplo 2:

```
struct Persona {
   char nombre[20], apellido[20];
   int edad;
};

int main(){
  Persona * ptr;
  ptr = new Persona;
  ......
```



A la variable ptr se le asigna la dirección de memoria de la variable dinámica de tipo Persona creada por el operador new.

Las variables dinámicas se acceden exclusivamente por punteros. No tienen un identificador asociado.

9

Asignación dinámica de memoria

- Cuando el programador necesite liberar la variable dinámica, deberá utilizar el operador:

delete

- Libera memoria asignada previamente por new
- El argumento para delete es un puntero a la dirección de memoria que se quiere liberar

Ejemplo:

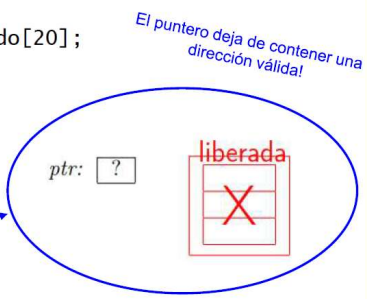
- delete Ptr;
- delete libera un único objeto.
- delete [ ] (con corchetes vacíos) libera un vector dinámico.
- C++ no realiza ninguna acción si el puntero que se le pasa a delete es nulo.
- Por esa razón se recomienda asignar cero o NULL al puntero inmediatamente después de usar delete. Se evita así que el puntero pueda ser usado de nuevo como argumento para delete.

10

Reserva dinámica de memoria

```
struct Persona {
   char nombre[20], apellido[20];
   int edad;
};

int main(){
  Persona * ptr;
  ptr = new Persona;
  ......
  ......
  delete ptr;
}
```



- *delete* libera la memoria dinámica reservada para la variable dinámica.
- La variable ptr queda con un valor no especificado. Los punteros que apuntan a memoria liberada deben ser manejados con cuidado, y lo más recomendable es asignar NULL al puntero, después de usar delete, para evitar problemas de punteros colgantes (dangling pointers).

11

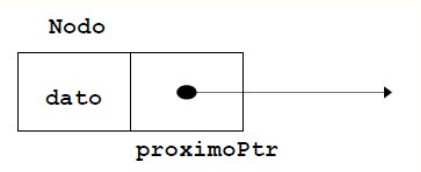
Estructuras autorreferenciadas

- Estructuras que tienen un puntero a una estructura del mismo tipo.
- Pueden ser enlazadas para formar estructuras de datos útiles como listas, colas, pilas y árboles.
- Terminan con un puntero NULL (0).

```
struct Nodo {
  int dato;
  Nodo *proximoPtr;
};
```

proximoPtr

- Apunta a un objeto de tipo nodo.
- Se conoce como un "enlace"
- enlaza un nodo a otro nodo.



12

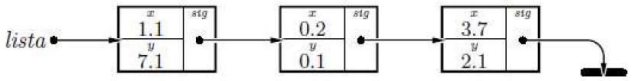
Estructuras autorrefenciadas

- Dos estructuras autoreferenciadas enlazadas juntas (formando una lista)



- Otro ejemplo: Una lista de puntos en el plano

```
struct Nodo {
   float x;
   float y;
   Nodo * sig;
};
```



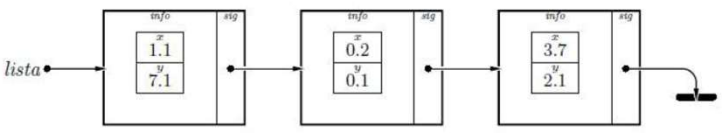
13

Estructuras autorrefenciadas

- Otra forma: se define un tipo adicional (struct Punto). Cada nodo utiliza un único campo de tipo Punto:

```
struct Punto {
   float x;
   float y;
};

struct Nodo {
   struct Punto info;
   struct Nodo * sig;
};
```



14

Listas Enlazadas

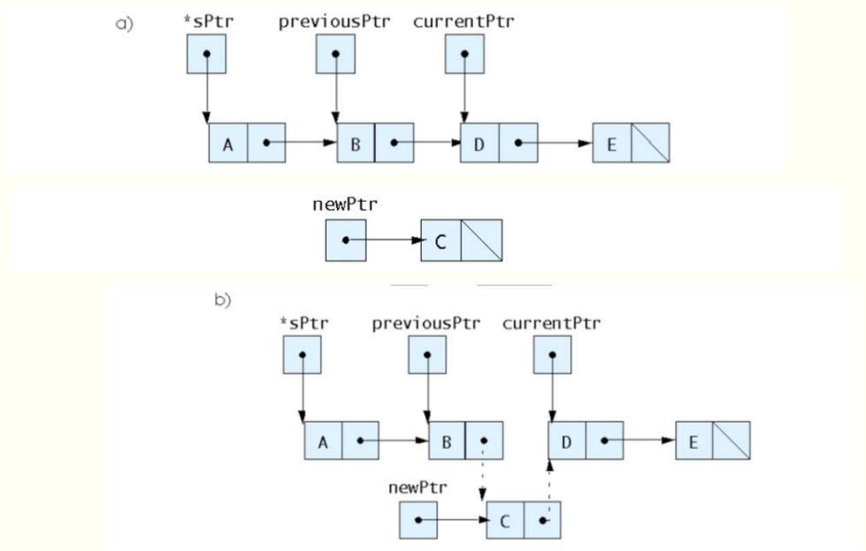
- Colección lineal de objetos autoreferenciados, llamados nodos.
- Conectados mediante punteros.
- Accedidos mediante un puntero al primer nodo de la lista (puntero maestro).
- Los nodos siguientes se acceden mediante el campo puntero (enlace) del nodo actual.
- El puntero enlace del último nodo se pone a NULL para marcar el final de la lista.
- Los datos se almacenan dinámicamente.
- Los nodos se van creando a medida que se los necesita.

Utilice una lista enlazada en lugar de un arreglo cuando:

- No se conozca por anticipado la cantidad de elementos de datos a representarse.
- Su lista necesite inserciones y eliminaciones rápidamente.
- La inserción y eliminación en arreglos ordenados insume mucho tiempo, ya que los elementos que se encuentran a continuación del elemento insertado/eliminado se deben desplazar adecuadamente.

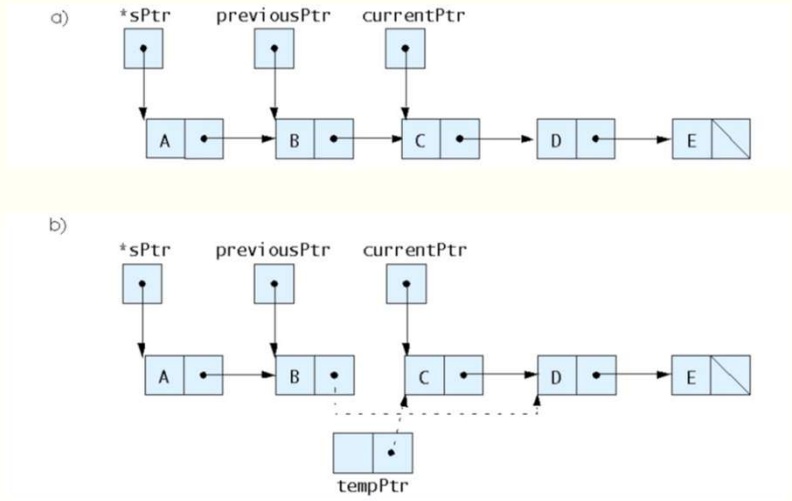
15

Listas Enlazadas: Inserción de un nuevo nodo



16

Listas Enlazadas: Eliminación de un nodo



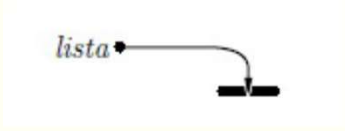
17

Listas Enlazadas

```
struct Nodo {
        int info;
        Nodo * sig;
        };
```

```
int main(){
    Nodo * lista = NULL;
```

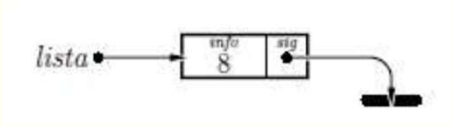
Crea la lista vacía




```
int main(){
    Nodo * lista = NULL;

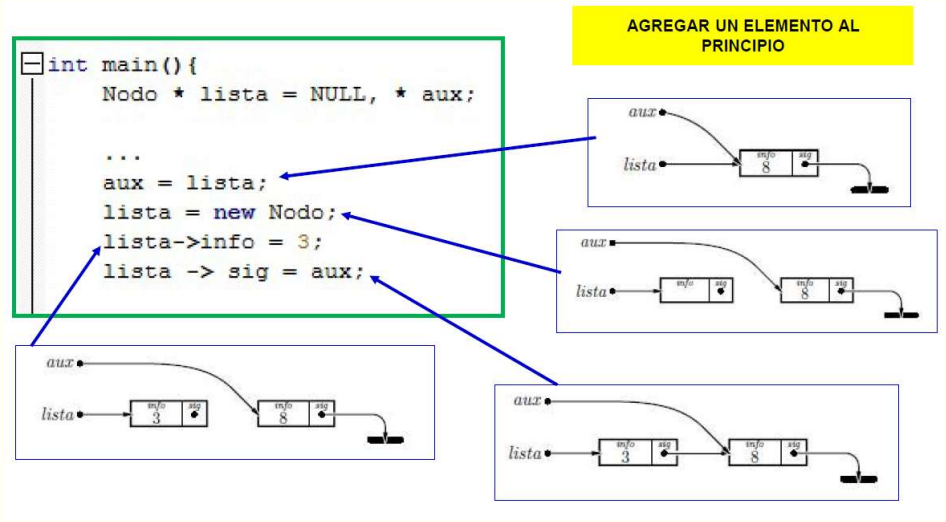
    lista = new Nodo;
    lista->info = 8;
    lista -> sig = NULL;
```

Crea el primer elemento



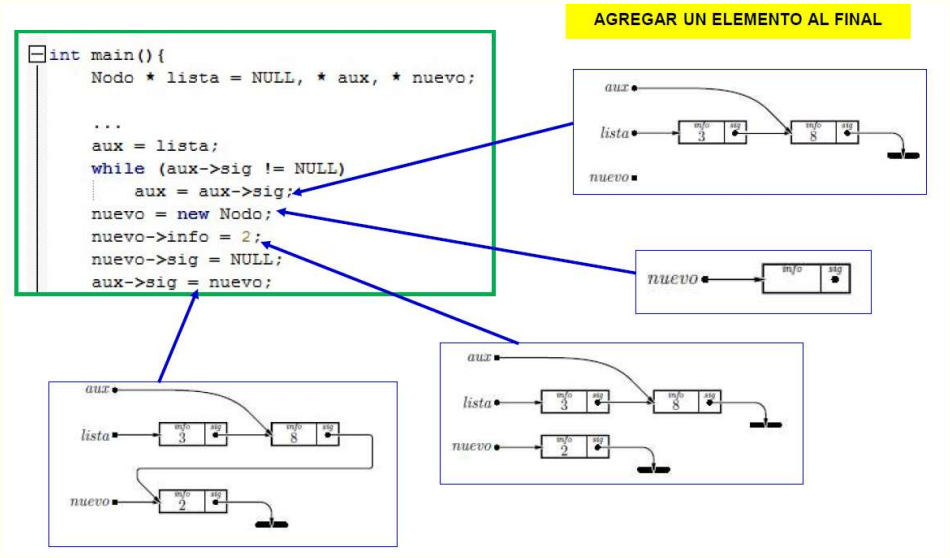
18

Listas Enlazadas



19

Listas Enlazadas



20

Listas Enlazadas

OTRA FORMA DE AGREGAR UN ELEMENTO AL FINAL

```
Nodo * lista = NULL, * aux, * nuevo;

...
for (aux = lista; aux->sig != NULL; aux = aux->sig);
nuevo = new Nodo;
nuevo->info = 2;
nuevo->sig = NULL;
aux->sig = nuevo;

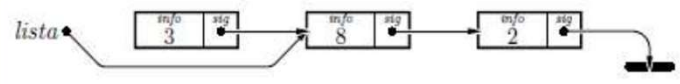
return 0;
```

Bucle *for* sin sentencia en su bloque. Sólo se limita a desplazar el puntero ***aux*** hasta que apunte al último elemento de la lista.

21

Listas Enlazadas: Borrar el primer elemento

```
1 int main(void)
2 {
3   struct Nodo * lista = NULL, * aux, * nuevo;
4
5   ...
6   lista = lista->sig; // ¡Mal! Se pierde la referencia a la cabeza original de la lista.
7
8   return 0;
9 }
```



Fugas de memoria: no se puede liberar un bloque de memoria.
Recolector de basura (Garbage Collector): mecanismo encargado de detectar porciones de memoria que no están en uso y liberarlas . C no lo implementa.

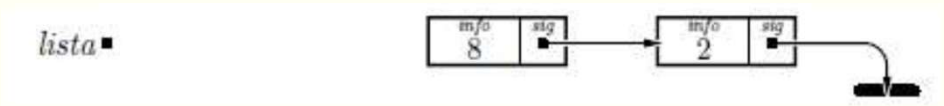
22

Listas Enlazadas: Borrar el primer elemento

```
int main(){
    Nodo * lista = NULL, * aux, * nuevo;

    ...
    delete lista;
    lista = lista->sig; // ¡Mal! lista no apunta a una zona de memoria válida

    return 0;
}
```



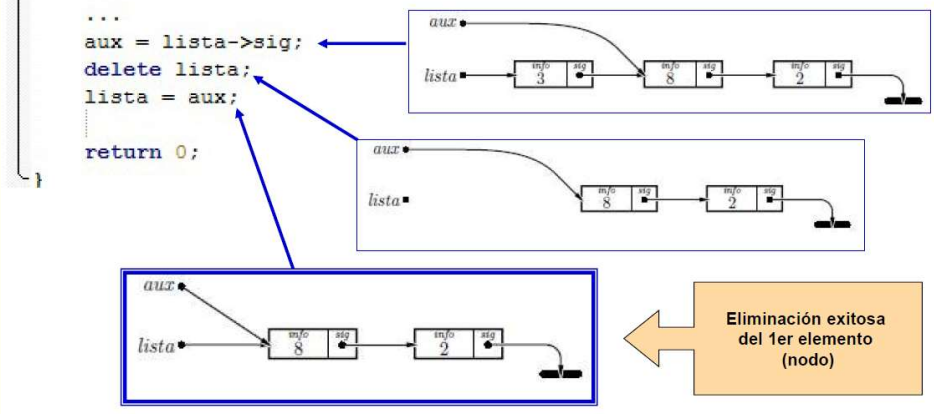
23

Listas Enlazadas: Borrar el primer elemento

```
int main(){
    Nodo * lista = NULL, * aux, * nuevo;

    ...
    aux = lista->sig;
    delete lista;
    lista = aux;

    return 0;
}
```



24

Listas enlazadas

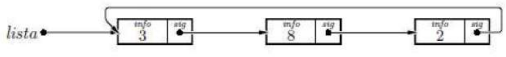
En el ejemplo que tienen busquen como agregar:

- Imprimir en Orden inverso la lista
- Encontrar el menor de una lista
- Ordenar una lista

25

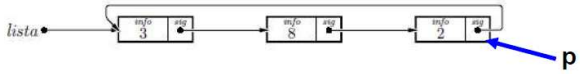
Otras listas

- **Listas circulares**
  - **Lista enlazada simple en la que el último nodo referencia al primero.**
  - **No hay un principio y fin definido.**
  - **Ningún nodo está enlazado a NULL.**
  - **El puntero se usa para reconocer un ingreso a la lista.**



  - **Como el último nodo tiene referencia al primero, se puede mantener un puntero (p) apuntando al último nodo y desde allí acceder al primero a través de p→sig.**

Así en una lista circular se tiene referenciado el último nodo e indirectamente el primero.

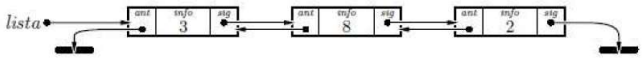


26

Otras listas

- **Listas doblemente enlazadas**

```
struct unnodo {
   int info;
    unnodo *ant;
    unnodo *sig;
};
```



Cada nodo tiene un puntero al siguiente y otro al anterior nodo

Se pueden mantener punteros al primero y al último nodo

Utiliza más memoria pero tiene las siguiente ventajas:

- La lista puede recorrerse en ambas direcciones
- Las operaciones insertar y eliminar utilizan menor cantidad de instrucciones

27

Algoritmos y Estructuras de Datos – AEDD – Asistencia 05/10

Contraseña del estudiante

h9zsg1

28

Para profundizar

Capítulo 13

"Memoria dinámica. Punteros"

Libro: "Fundamentos de Programación con el Lenguaje de Programación C++" – Benjumea y Roldán, pág. 163

"Punteros y datos dinámicos", pág. 895

"Listas Enlazadas", pág. 964

Libro: "Fundamentos de la programación" – Yáñez,

29

Algoritmos y Estructuras de Datos – AEDD – Agenda 05/10

- ✓ Estructuras de datos dinámicas.
- ✓ Asignación dinámica de memoria
- ✓ Reserva dinámica de memoria.
- ✓ Estructuras autorreferenciadas.
- ✓ Lista enlazada: Agregar un elemento al final, agregar un elemento al principio.
- ✓ Lista enlazada: Borrar el primer elemento.
- ✓ Lista enlazada: Ejemplo
- ✓ Otras listas

¡OBJETIVO CUMPLIDO!

30

Para profundizar

Capítulo 12

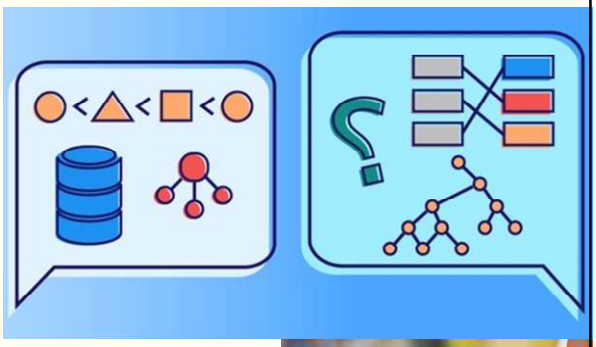
"Apuntadores"

Libro: "C++ para ingeniería y ciencias" 2da edición -

Gary J. Bronson, pág. 667

31

ALGORITMOS Y ESTRUCTURA DE DATOS



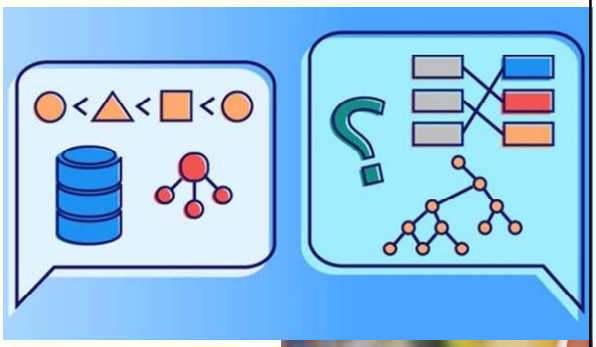
Comisión F – 2026 – Cursado Anual

¿DUDAS?



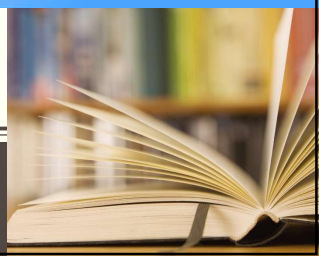
32

ALGORITMOS Y ESTRUCTURA DE DATOS



Comisión F – 2026 – Cursado Anual

¡Muy buena semana!



33